ERS O&M Portal — Read Me First

Updated 14 October 2026. Start with this page. It is the front door to the handover.

This is a **working prototype**, not a finished system. It runs and the main flows work, but several parts are simulated (section 6). Continue from this code, and read sections 6 to 8 before you trust any screen.

1. What is in the handover

Item	What it is for	Read it when
This page (Read Me First)	The big picture, how to run it, what is fake, what is open	First
Data Dictionary	Every field in `db.json`, with allowed values	Before you design the database
API Endpoint Reference	Every call the server accepts, who may make it, and what it returns	Before you write the new back end
The code (`server.py`, `static/`)	The working prototype	After the three pages
The project sheet (Google Sheet)	The list of features and jobs, one row per item	To match a screen to a requirement
System Blueprint	The first design document, dated 3 October	Only for history. It is older than the code.

Where a document and the code disagree, **the code is right** about what the prototype does today. Tell us if you find a difference.

2. The system in one minute

The ERS O&M Portal tracks service jobs for solar systems, from a customer problem to a closed ticket.

- A **ticket** is one job. It has a type, a client, a visit date and a report.

- A **CS** person creates the ticket and books the visit. A **technician** (or an engineer going alone) visits the site, fills a **safety checklist** and then the **report**. An **engineer** reviews and approves the report. The **HOD** or CS closes the ticket.
- Everyone sees the same live data, in a web page that suits their role. Technicians use a phone layout.

How a ticket moves

Step	State	What happens	Who does it
1	`created`	Ticket is made	HOD, Engineer, CS
2	`scheduled`	A calendar event is linked, and people are chosen	HOD, CS
3	`site_visit`	The first checklist is opened on the visit day	Technician, or Engineer on an engineer-only visit
4	`report_drafting`	Every visit day has a submitted checklist (or the day passed). Moves on its own.	automatic
5	`engineer_review`	The report is submitted	Technician or Engineer
6	`approved`	The report is approved. A reject sends it back to step 4.	HOD or Engineer
7	`closed`	The ticket is closed	HOD, CS

A ticket can also be closed early (any step before 6, with a reason), and a ticket in step 1 can be deleted. A closed ticket cannot be changed any more, except for messages.

Who can do what

What	HOD	Engineer	CS	Technician
Create a ticket	Yes	Yes	Yes	–
Schedule a visit	Yes	–	Yes	–
Fill the safety checklist	–	Yes (own solo visit)	–	Yes (own team)

What	HOD	Engineer	CS	Technician
Fill and submit the report	–	Yes	–	Yes
Approve or reject the report	Yes	Yes	–	–
Close or delete a ticket	Yes	–	Yes	–

The full table, including the smaller actions, is in the API Endpoint Reference, section 4. An engineer may approve the report they wrote themselves; the HOD agreed to this. When an engineer goes to site alone, the safety checklist is mandatory for them.

3. Run it

The system is plain HTML and JavaScript pages served by one Python script. It needs Python 3.8 or newer and no other packages. It has been run on Python 3.11.

1. Open a terminal in the project folder (the one that contains `server.py`).
2. Run `python3 server.py` (on Windows: `python server.py`).
3. Open `http://localhost:8000/` and click one of the 9 demo accounts. There is no password.

To stop the server press Ctrl+C.

What is in the folder

Item	What it is
`server.py`	The whole back end: the API and every rule
`db.json`	The only database. Every change rewrites the whole file. It holds demo data.
`static/`	All pages (one `.html` file per screen), `shared.js` (code every page uses), `style.css`, `sw.js` (offline helper), `manifest.json`
`static/icons/`, `static/kb-assets/`	Pictures used by the app icon and the Knowledge Base. Keep them with the code.
`backups/`	Made by the server: one copy of `db.json` per Malaysian day, newest 14 kept

Item	What it is
`om_actions.log`	Made by the server: one line per ticket action, successful or not
`om_email_outbox.log`	Made by the server: the mock email for each @mention

Settings can be changed without editing code, using environment variables:

Setting	Default	Meaning
`OM_PORT`	8000	Port
`OM_DB_PATH`	`db.json` next to `server.py`	Where the data file is
`OM_STATIC_DIR`	`static` next to `server.py`	Where the pages are
`OM_LOG_PATH`	`om_actions.log`	Action log
`OM_EMAIL_LOG_PATH`	`om_email_outbox.log`	Mock email log
`OM_TZ_OFFSET_HOURS`	8	Hours added to UTC to get "today". The pages have their own copy of this number in `static/shared.js` (`BUSINESS_UTC_OFFSET_HOURS`). **Change both together.**

Demo accounts: HOD - Steve; Engineer - Wardah and Engineer - Faisal; CS - Alice and CS - Mei; Technician - Ali and Bala (Team 1); Technician - Chong and Devi (Team 2).

The pages ask the server for fresh data every 4 to 8 seconds. There is no live push. When a page file such as `shared.js` changes, browsers pick it up on the next load; no hard refresh is needed.

4. Where things are in the code

Pages (`static/`)

Page	For	What it shows
`index.html`	All	Pick an account
`dashboard-admin.html`, `dashboard-cs.html`, `dashboard-engineer.html`, `dashboard-technician.html`	One role each	The home screen
`admin.html`, `cs.html`, `engineer.html`	HOD, CS, Engineer	The ticket list
`ticket-new.html`	HOD, Engineer, CS	Create a ticket
`ticket-detail.html`	HOD, Engineer, CS	One ticket: schedule, thread, report summary, close
`job-detail.html`	Technician, Engineer	The visit and its steps. On a wide screen an engineer gets the desktop layout.
`checklist-list.html`, `checklist.html`	checklist-list: Technician. checklist: any role can open it, only Technician and Engineer can fill it	Safety checklist
`report-list.html`, `report.html`	report-list: Technician. report: any role can open it, only Technician and Engineer can edit it	Fill the report. One long scrolling page with one Save all button.
`reports.html`, `safety-checklists.html`	reports: HOD, Engineer, CS. safety-checklists: any role	Lists of reports and checklists
`calendar.html`, `technician-calendar.html`	calendar: HOD, Engineer, CS. technician-calendar: Technician	Calendar
`clients.html`, `client-detail.html`	Staff	Clients and sites

Page	For	What it shows
`tools.html`	HOD, Engineer, CS	Tools
`analytics.html`	HOD	KPIs and charts
`knowledge-base.html`	HOD, Engineer, CS	Articles
`users.html`	HOD, CS	Technician teams

Code to read first

- `static/shared.js` holds what every page uses: the page frame and sidebar (`renderShell`, `renderMobileShell`), the call to the server (`api`, `doAction`), the role table (`ACTION_ROLES`, `canDo`), "today" (`todayYmd`), SLA badges (`slaInfo`) and the Analytics sums (`departmentSummary`, `kpiSummary`).
- `server.py` holds every rule. Read in this order: the settings at the top, `ACTION_ROLES`, `ACTIONS`, the closed-ticket lock (`ACTIONS_ALLOWED_ON_CLOSED_TICKET`) and the report lock (`REPORT_LOCKED_STATUSES`).
- In the project sheet, the Category column names the tab a row belongs to, so you can match a row to a page.

5. What the server enforces

These rules live in `server.py`, not only in the pages, so a page cannot skip them.

- **Roles.** Every ticket action checks the role of the person named in `actor` (403 if not allowed).
- **Checklist team rule.** A technician fills a checklist only for their own team's visit day. An engineer only on a day with no technician where they are the engineer.
- **Checklist day.** A checklist can only be filled on its visit date, in Malaysia time, and never after submit. A failed item needs a note or a photo label.
- **Closed ticket lock.** After closing, only messages and a few HOD or CS actions are allowed.
- **Report lock.** Once a report is submitted or approved it cannot be edited. A reject opens it again.
- **Input checks.** A new ticket needs a real client and a valid type and priority. Wrong types of values return a clean 400 and save nothing.

- **Text clean-up.** `<`, `>` and `"` in typed text are replaced before saving, so typed text cannot run as code (Knowledge Base text excepted; its page escapes it).
- **Request size.** Bodies over 5 MB are refused.
- **Safe saving and backup.** `db.json` is saved through a temporary file, so a crash cannot leave a half file. One copy per Malaysian day goes to `backups/` (newest 14 kept).
- **Malaysia time.** "Today" follows UTC+8 on both the server and the pages.

6. What is simulated

These look finished on screen but are not real. Replace them before real users depend on the system.

Area	What the prototype does	What is needed	Priority
Login	You pick an account. No password. The browser sends a name with each action and the server believes it. Anyone can send another person's name. The server also listens on all network addresses.	Real login. Take the user from the session on every request.	**First**
Database	One `db.json`, rewritten in full on each change, one request at a time. In a test with 30 people clicking at once, the slowest replies took about 2 seconds.	A real database	High
Photos and files	Only a text label is saved. No file is stored. The NTP PDF upload works the same way.	Real file storage	High
Email	An @mention writes a line to `om_email_outbox.log`. Nothing is sent.	A real email service	Medium
Google Calendar	A form on the Calendar page stands in for the one-way sync.	Real sync	Medium
Backup	The daily copy sits on the same disk as `db.json`.	Also copy it to another machine	High

Area	What the prototype does	What is needed	Priority
Notifications	The bell shows @mentions only.	Decide whether to add live updates	Low
Calibration Tracking, Activity Log	Shown in the sidebar as coming soon.	Not built	–

New users should also note that all reading (GET) calls have no permission check. Anyone who can reach the server can read everything.

7. What is not built yet

- **Report formats.** The code has two: Residential (Format A) and Troubleshoot (Format B). Every other type, including C&I Servicing, Site Survey, DERE and NTP Phase 1 to 4, gets Format A. The project sheet (row 111) lists six formats, so four are still to be built.
- **KPIs that need new data.** MTTR needs a fault start time and a restored time. MTTD needs alerts from a monitoring system. Contract SLA needs the SLA period stored per client or site. First-Time Fix Rate needs a definition of "fixed". Sheet row 18c has the detail.
- **Monthly trend of the KPIs.** Analytics can switch between Last 30 days, This month, Last 3 months and This year, but the demo data covers only a few weeks, so longer periods look the same as 30 days.
- **A way to add a site to a client.** No call does it. See the Data Dictionary, section 8.

8. Decisions still open

The prototype contains our best guess for each. Confirm them with the HOD before you build on them.

Decision	What the prototype does now	Who confirms
Safety checklist and report for Residential Troubleshoot/Rectification	**Settled on 14 October.** The HOD confirmed that this type uses the **residential** checklist questions, with the **Troubleshoot (Format B)** report. This is what the code does, so nothing needs to change in the code. C&I questions are used only for C&I Servicing, C&I Troubleshoot, DERE and Site Survey. The project sheet (rows 44 and 111) is being updated to match.	HOD
Preventive versus Corrective	Servicing types count as Preventive. Troubleshoot/Rectification and DERE count as Corrective. Site Survey and NTP are not counted.	HOD
First-Time Fix Rate	Not built. We need a definition of when a job counts as fixed on the first visit.	HOD
KPI benchmarks	The target numbers on the HOD's slide are unverified. Ask where they come from before you draw target lines.	HOD
Internal SLA limits	Engineer review within 3 days and CS closure within 2 days. These came from the prototype, not a contract.	HOD
Project dates	The dates in the sheet are draft. The planned window is early November to mid December.	HOD
Linking a new event to a ticket that is already in review or approved	The ticket drops back to `scheduled` but the report keeps its status. Nobody has decided if this is right.	HOD

9. Known limits and small issues

Found by reading the code and by testing, and checked again on the final version on 14 October.

- **Old ticket types.** `servicing` and `troubleshooting` can no longer be chosen, but about half of the demo tickets still use them. They get the Residential report, even `troubleshooting`. Move them to the new types when you migrate.
- **Wrong-format report actions.** A report action meant for the other report format returns 500, not 400. The pages never send these.
- **`report_section` accepts any field names.** It does not check what is inside the `patch`.
- **Knowledge Base built-in articles.** Changing the audience of an article that is built into the page is not saved, so it resets on reload.
- **Analytics.** Technician cards ignore engineer-only (solo) visits.
- **Wide lists.** `GET /api/tickets` sends every ticket with its full report, so it will slow down as data grows.

10. What was tested, and what was not

Tested	How
Every ticket action and its error cases	Automated calls against a running server
Role rules for each role	Calls with each role's name, expecting 403 where the table says no
The pages	Opened in a desktop browser (Chromium) as each role, clicking through the main flows
Typed text that tries to run as code	Sent through the API, then the pages that show it were loaded and none ran it
Many users at once	A simulated burst of about 30 users
The real data	A full check of the real project folder and data: every file matched, every page loaded, every ticket read cleanly

Not tested	Why it matters
A real phone	The phone layouts were checked at phone size in a desktop browser only
A real Malaysian clock	The time rule was checked by reading the code and by tests, not by using the system across a real Malaysian midnight
Real Google Calendar, email, file upload	They are not built
Real users over several days	Nobody has used it in daily work yet

11. Suggested order of work

This is our suggestion. Change it as you see fit.

1. Real login and server-side identity. Everything else assumes the user is who they say they are.
2. A real database in place of `db.json`, designed from the Data Dictionary.
3. Email, Google Calendar and file upload.
4. The four missing report formats.
5. The KPIs that need new data, then the monthly trend.

12. Notifications: what to build

Written from `server.py`, `shared.js`, `sw.js` and `manifest.json` on 15 October 2026. It was checked by reading the code, not by running a phone.

12.1 What exists today

- **One kind of notification only: an @mention in a ticket message.** The bell icon in the top bar shows a red badge and a dropdown. The page asks the server for the tickets once, when it loads, and works out the unread mentions itself. The badge does not update by itself while the page stays open. Technicians do not get the bell, because they cannot use the ticket thread.

- **A mock email for the same @mention.** The server writes one line to a log file (`OM_EMAIL_LOG_PATH`). It uses the person's **name**, because accounts have no email address. No email is sent.
- **The phone app shell is ready.** `manifest.json`, the icons, `sw.js` (offline page, update banner) and the install banner in `shared.js` are in place. The portal can be installed on Android, iPhone (Add to Home Screen) and a computer.
- **Nothing else notifies anyone.** Assigning someone, scheduling a visit, a reschedule request, a submitted or rejected report and closing a ticket do not notify the people involved. A person sees them only when they open the portal.

12.2 What is missing

Part	Missing
Email	Real sending (SMTP or an email service), and an email address on every account.
Phone push	A push handler in `sw.js`, asking the user for permission, storing each user's push subscription, and sending from the server.
Events	The server does not raise a notification for any event except an @mention.
Timed reminders	There is no background job. See 12.4.

12.3 Events and who should be told (proposal)

The code already records each of these events in the ticket's activity log (`event_log`), so the digital team can raise a notification at the same place. The recipients below are a **proposal for the HOD to confirm**, not something the code does today. Technicians are not on a ticket's assignee list; they are found on the visit days of the ticket (`visit_days[].technicians`). The engineer of a visit is `visit_days[].engineer`. The person who submitted a report is `report.submitted_by`.

Event (name in `event_log`)	Who to tell	Channels
@mention in a message (exists today)	The mentioned person	Bell, email, push

Event (name in `event_log`)	Who to tell	Channels
A person is added to a ticket (`assign_add`)	That person	Bell, email, push
A visit is scheduled or changed (`link_events`, `day_people`)	Engineer and technicians of that visit day	Bell, email, push
A visit is removed (`unlink_event`)	Engineer and technicians who were on that day	Bell, email, push
A technician requests a reschedule (`reschedule_request`)	The ticket's CS and the HOD	Bell, email, push
A reschedule request is cleared (`reschedule_cleared`)	Engineer and technicians of the ticket	Bell, push
A report is submitted (`report_submitted`)	The ticket's engineer, and the HOD	Bell, email, push
A report is rejected (`report_rejected`)	The person who submitted it, and the technicians of the ticket. Include the reason.	Bell, email, push
A report is approved (`report_approved`)	The ticket's CS (who closes it), and the HOD	Bell, email
A ticket is closed or closed early (`close`, `early_close`)	Everyone assigned to the ticket	Bell
A visit day passes unfinished (`day_unfinished`, automatic)	The ticket's engineer, CS and HOD	Bell, email

The digital team can decide to leave some of these out. Too many notifications make people ignore them.

12.4 Reminders that need a scheduler

Some useful notifications are about time passing, not about an action. The server has **no background job**. A few automatic steps only run when someone reads or acts on a ticket (for example `reconcile_visit_days`, which marks a day unfinished and moves the ticket into report drafting). So these need a scheduler (cron, a task queue or similar) that the digital team adds:

- A ticket's snooze date arrives (`snooze_until`).

- A visit is tomorrow or today.
- SLA warnings: the engineer has 3 days to review a report, and CS has 2 days to close after approval. The dashboards show these today but nothing is sent.
- A ticket was created and never scheduled for more than 7 days.

12.5 How to build it, in order

1. **Add an email address to every account.** Without it nothing can be sent. The digital team's own login system probably has this already.
2. **One function that sends a notification** (who, event, ticket, text). It looks up the person's address and push subscription, and sends. Replace `queue_mock_email()` and call the new function from the same place. Do not let a failed send break the user's action; the old code already works this way.
3. **Call it from each event in 12.3.** Do it in the same place the event is written to `event_log`.
4. **Email first.** It works on every phone and needs no HTTPS-only setup. Use the company's SMTP or an email service.
5. **Then phone push (Web Push).** Add a `push` handler in `sw.js` that shows the notification, a button or prompt that asks for permission after login, a table that stores each user's subscription, and server code that sends the push. This needs HTTPS. On iPhone it works only after the app is added to the Home Screen (iOS 16.4 or newer).
6. **Then the reminders in 12.4**, once a scheduler exists.
7. **Let each user choose.** At least turn email or push off per user, and keep a log of what was sent.

12.6 Things to decide

- Which events in 12.3 really need email and push (HOD).
- Whether WhatsApp or Telegram is wanted as another channel. WhatsApp Business is paid and needs approval; a Telegram bot is free.
- Whether to send at night, or hold messages until morning.